

Professional Guide to Git Branching Merging

January 17, 2026

1 Introduction to Branching

Branching allows developers to diverge from the main line of development and continue to work without messing up the main line. It is conceptually similar to creating a copy of your version, where you can add commits to the copy instead of the main version of the repository.

1.1 The Use Case

A common scenario involves fixing a bug while simultaneously adding new features:

1. You are on the `master` branch.
2. You need to work on a new feature, so you branch off to `feature1`.
3. Suddenly, a bug is found in production. You switch back to `master` or create a `hotfix` branch to fix the bug.
4. Once fixed, you continue working on `feature1`.

2 Basic Branching Operations

To create a branch, we use the command `git branch <branchname>`. The branch name is usually descriptive of the feature being developed.

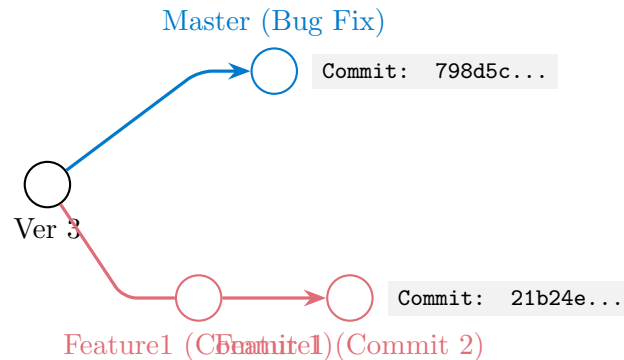
```
# Create a new branch called feature1
skysys@Skysys:~$ git branch feature1

# Switch to the new branch
skysys@Skysys:~$ git checkout feature1
```

2.1 Visualizing the Divergence

When changes are made in the `feature1` branch and subsequently a bug fix is applied to the `master` branch, the history splits. The `master` branch moves forward with the fix, and the `feature1` branch moves forward with the feature.

Visual Representation: Diverged History



In the log output, this appears as:

```
* commit 798d5c7... (HEAD -> master)
|   Bug Fix
|
| * commit 21b24e2... (feature1)
| |   Feature Commit 2
| |
| * commit fc8ee08...
|/
```

3 Merging

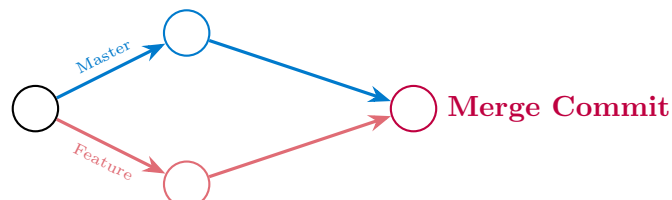
Combining different branches together is called **merging**. The result of a merge is applied to the branch currently checked out.

To merge `feature1` into `master`:

```
git checkout master
git merge feature1 -m "Merge branch feature1"
```

3.1 Visualizing the Merge

When the merge occurs, Git creates a "Merge Commit" that has two parents: the tip of the `master` branch and the tip of the `feature1` branch.



4 Merge Conflicts

Merge conflicts occur when Git cannot automatically determine how to combine changes. This typically happens when the same line of code is modified differently in two distinct branches.

4.1 Example Scenario

Consider the following JavaScript function modified in two branches:

Branch 1 (Master)	Branch 2 (Feature)
<pre>function a() { // return 5; alert(5); }</pre>	<pre>function a() { // return 5; confirm(5); }</pre>

Table 1: Conflicting Code Changes

Git enters a conflict state and pauses the merge. The file will contain conflict markers:

```
<<<<<< HEAD  
    alert(5);  
=====  
    confirm(5);  
>>>>>> feature1
```

Resolution: You must manually edit the file to choose the desired code (or combine them), remove the markers, and then commit the changes.

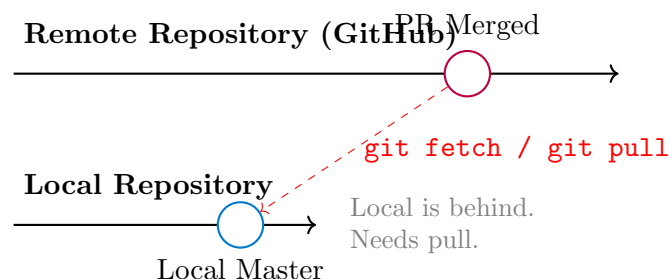
5 Feature Branch Workflow & Collaboration

In a professional environment, code is reviewed via Pull Requests (PRs) on platforms like GitHub before merging.

5.1 Workflow Steps

1. **Create Feature Branch:** `git branch feature-new`
2. **Push to Remote:** Upload the branch so others can see it.
3. **Pull Request (PR):** Request the admin to merge your feature into `master`. This initiates a Code Review.
4. **Syncing Local Repo:** After the PR is merged remotely, your local `master` is out of date.

5.2 Visualizing Remote Synchronization



Command Summary

- `git branch <name>`: Create a branch.
- `git checkout <name>`: Switch branches.
- `git merge <name>`: Merge `name` into current branch.
- `git fetch`: Update remote tracking branches.
- `git pull`: Fetch and merge remote changes.